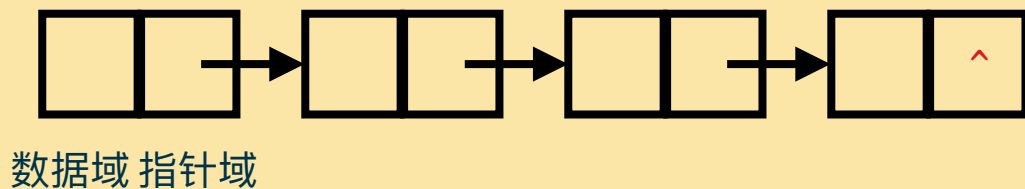




1. 线性表的链式表示

①单链表的定义



数据域 存放数据元素的信息

指针域 存放后继节点的地址

单链表不再依赖连续内存空间，但每个节点需要存储一个额外的指针，增加了存储开销

单链表不支持**随机存取**



1. 线性表的链式表示

①单链表的定义



指针本身也是一个存着数据的变量（对象）
只不过它存的数据是地址
也就是这个指针指向节点的地址

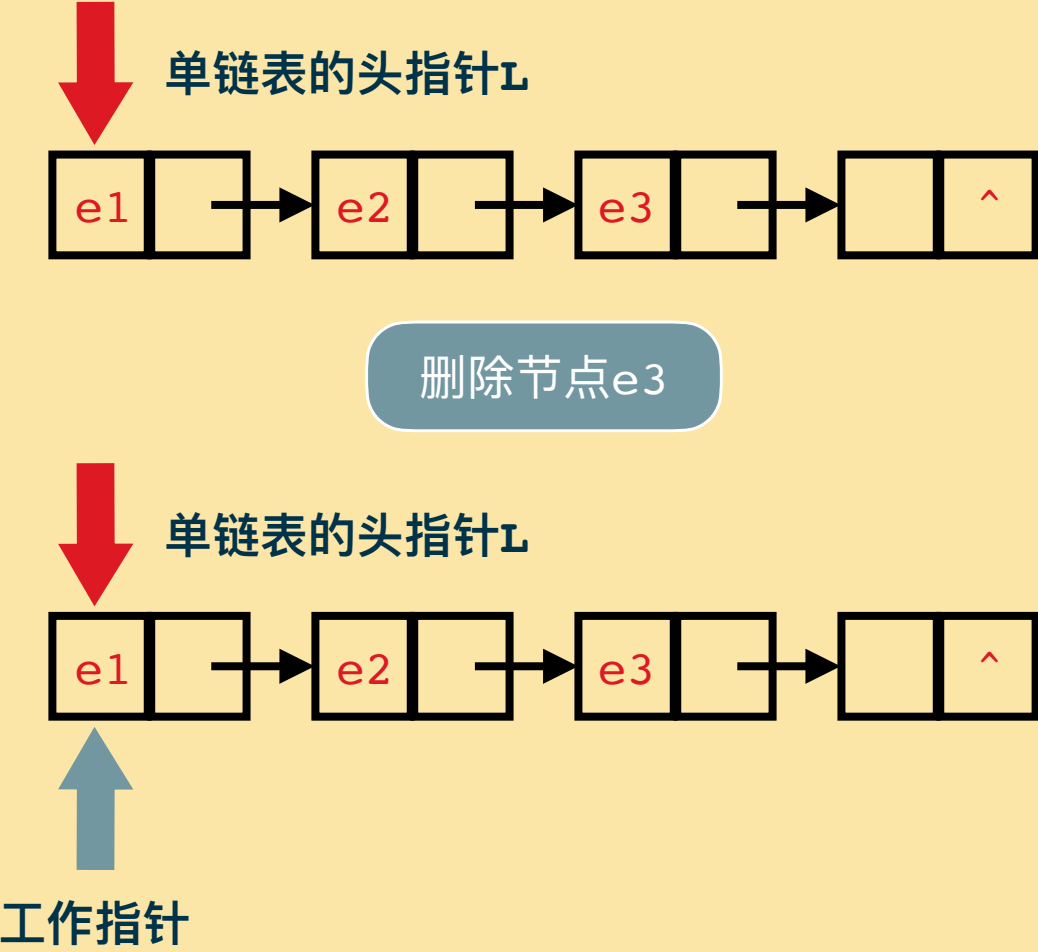
数据域 存放数据元素的信息

指针域 存放后继节点的地址



1. 线性表的链式表示

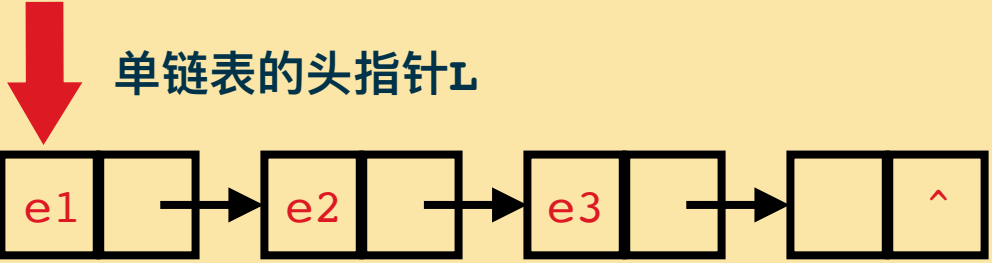
①单链表的定义



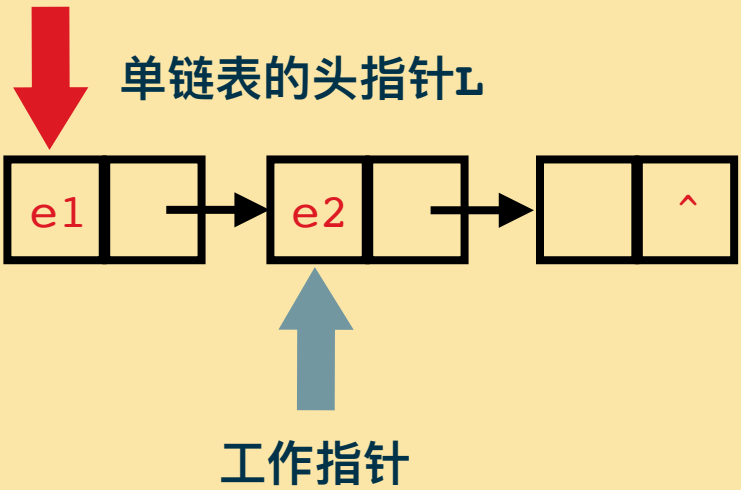


1. 线性表的链式表示

①单链表的定义



删除节点e3

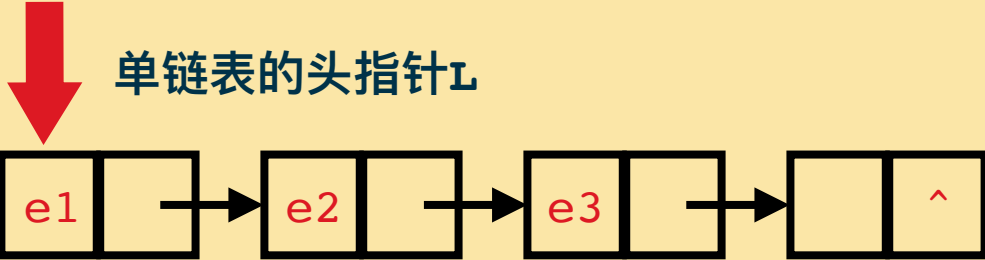


- (1) 将工作指针移动到待删除节点e3的前驱节点上
- (2) 修改前驱节点的next指针域，使其指向待删除节点e3的后继节点
- (3) 删除节点e3，释放内存空间

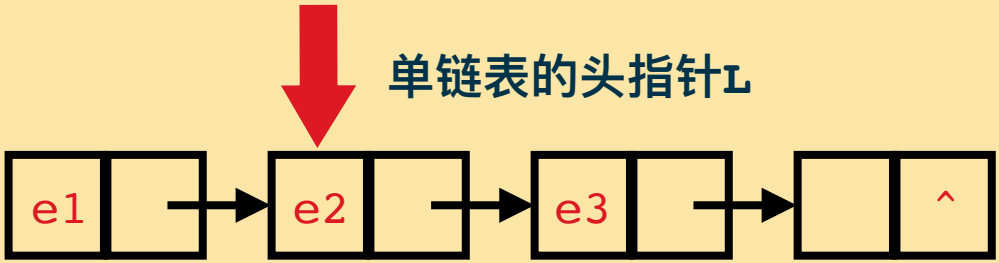


1. 线性表的链式表示

①单链表的定义



删除节点e1



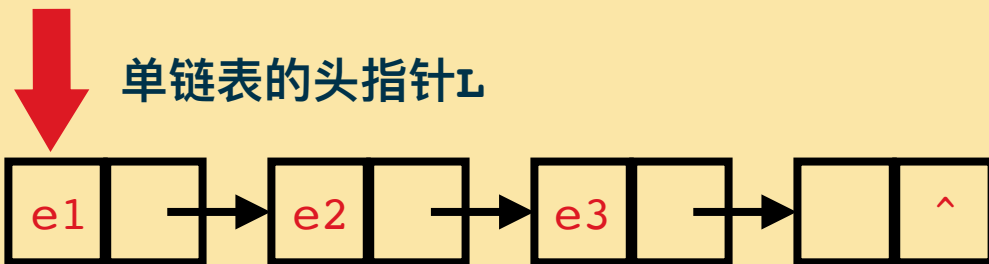
工作指针

- (1) 将工作指针移动到待删除节点e1上
- (2) 将头指针L后移一个节点
- (3) 删除节点e1，释放内存空间



1. 线性表的链式表示

①单链表的定义



删除节点e3

- (1) 将工作指针移动到待删除节点e3的前驱节点上
- (2) 修改前驱节点的next指针域，使其指向待删除节点e3的后继节点
- (3) 删除节点e3，释放内存空间

删除节点e1

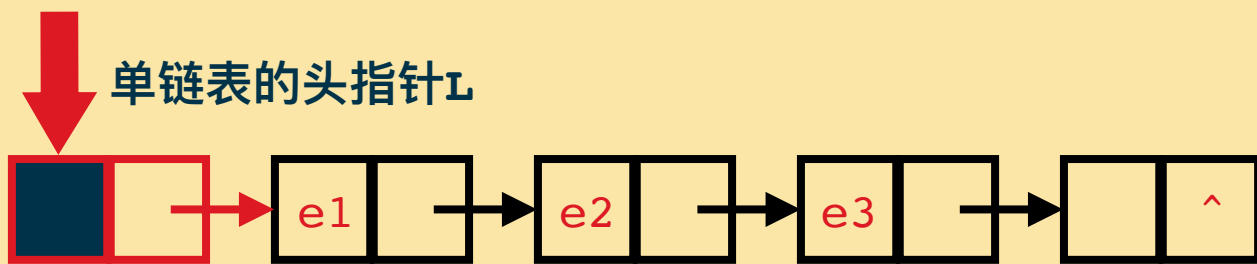
- (1) 将工作指针移动到待删除节点e1上
- (2) 将头指针L后移一个节点
- (3) 删除节点e1，释放内存空间

删除不同位置的元素时，操作并不统一



1. 线性表的链式表示

①单链表的定义

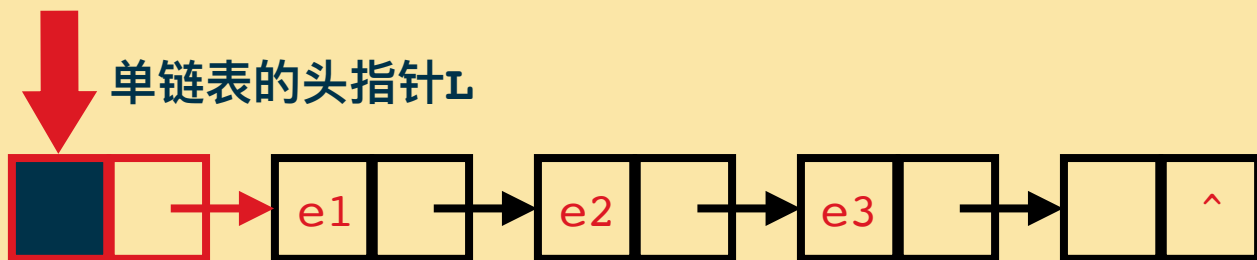


解决方案：增设一个不存任何数据元素的**头节点**



1. 线性表的链式表示

①单链表的定义



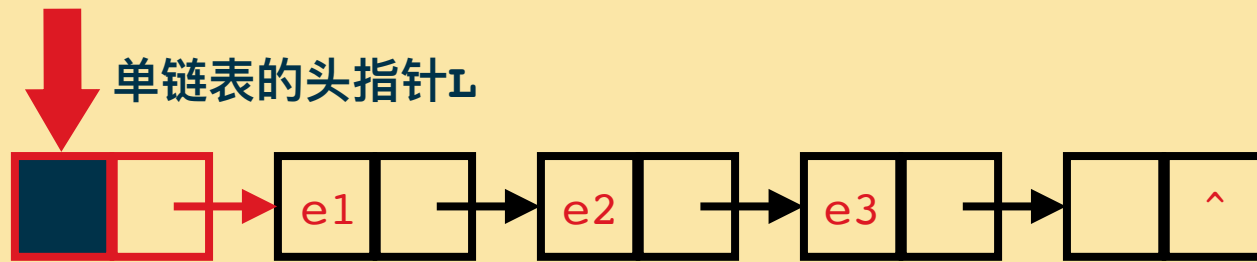
删除节点e

- (1) 将工作指针移动到待删除节点e的前驱节点上
- (2) 修改前驱节点的next指针域，使其指向待删除节点e的后继节点
- (3) 删除节点e，释放内存空间



1. 线性表的链式表示

①单链表的定义



解决方案：增设一个不存任何数据元素的**头节点**

头节点的作用：便于首元节点的处理



1. 线性表的链式表示

①单链表的定义

```
typedef struct LNode{  
    ElemType data; //节点的数据域  
    struct LNode *next; //节点的指针域  
}LNode, *LinkList;
```



1. 线性表的链式表示

①单链表的定义

```
typedef struct LNode{  
    ElemType data; //节点的数据域  
    struct LNode *next; //节点的指针域  
}LNode, *LinkList;
```

LNode

表示这种节点类型本身

工作指针、遍历指针
临时指针等

***LinkList**

表示指向这种节点的指针类型

头指针



1. 线性表的链式表示

②单链表的基本操作

初始化

```
bool InitList(LinkList &L)
{
    L = (LNode*)malloc(sizeof(LNode)); //创建头节点
    L->next = NULL; //头节点后暂无数据节点
    return true;
}
```



1. 线性表的链式表示

②单链表的基本操作

求表长

```
int Length(LinkList L)
{
    int len = 0; //记录表长的变量，初值为0
    LNode *p = L;
    while(p->next != NULL)
    {
        p = p->next;
        len++; //每访问一个节点，计数值+1
    }
    return len;
}
```



1. 线性表的链式表示

②单链表的基本操作

按序查找

```
LNode *GetElem(LinkList L, int i)
{
    LNode *p = L; //设置工作指针p
    int j = 0; //变量j用于记录当前位序
    while(p != NULL && j < i) //循环找到第i个节点
    {
        p = p->next;
        j++;
    }
    return p; //返回指向第i个节点的指针或NULL
}
```



1. 线性表的链式表示

②单链表的基本操作

按值查找

```
ListNode *LocateElem(LinkList L, ElemType e)
{
    ListNode *p = L->next; //设置工作指针p
    while(p != NULL && p->data != e) //从首元节点开始，循环找到数据域为e的节点
        p = p->next;
    return p; //找到后返回指向该节点的指针或NULL
}
```



1. 线性表的链式表示

②单链表的基本操作

插入

```
bool ListInsert(LinkList &L, int i, ElemType e)
{
    LNode *p = L; //设置工作指针p
    int j = 0; //变量j用于记录当前位序
    while(p != NULL && j < i-1) //循环找到第i-1个节点（待插入位置的前驱）
        p = p->next;
        j++;
    LNode *s = (LNode*)malloc(sizeof(LNode)); //申请新节点s作为插入的节点
    s->data = e; //将数据元素e放在待插入节点s的数据域
    s->next = p->next; //修改待插入节点s的next域
    p->next = s; //修改待插入位置前驱节点的next域
    return true;
}
```



1. 线性表的链式表示

②单链表的基本操作

删除

```
bool ListDelete(LinkList &L, int i, ElemType &e)
{
    LNode *p = L; //设置工作指针p
    int j = 0; //变量j用于记录当前位序
    while(p != NULL && j < i-1) //循环找到第i-1个节点（待删除节点的前驱）
        p = p->next;
        j++;
    LNode *q = p->next; //设置工作指针q指向待删除节点
    e = q->data; //取出待删除节点的数据域，用e返回
    p->next = q->next; //修改待删除节点的前驱节点的next域
    free(q); //释放待删除节点的存储空间
    return true;
}
```



1. 线性表的链式表示

②单链表的基本操作

头插法

```
LinkList List_HeadInsert(LinkList &L)
{
    LNode *s; int x;
    L = (LNode*)malloc(sizeof(LNode)); //创建头节点
    L->next = NULL; //初始将链表置空
    scanf("%d", &x); //输入节点的数据域
    while(x != 9999){ //输入9999表示插入结束
        s = (LNode*)malloc(sizeof(LNode)); //申请新节点s作为插入的节点
        s->data = x; //将数据元素x放在待插入节点s的数据域
        s->next = L->next; //修改待插入节点s的next域
        L->next = s; //修改头节点的next域
        scanf("%d", &x);} //继续输入下一个节点的数据域
    return L;
}
```



1. 线性表的链式表示

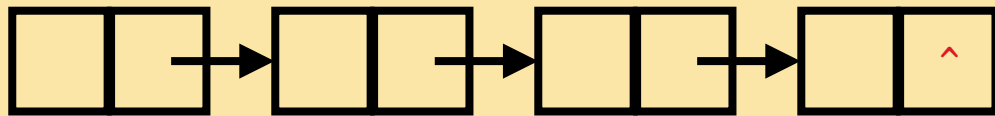
②单链表的基本操作

尾插法

```
LinkList List_TailInsert(LinkList &L)
{
    int x; LNode *s, *r = L; //设置指针s指向即将插入的新节点, 指针r指向尾节点
    L = (LNode*)malloc(sizeof(LNode)); //创建头节点
    L->next = NULL; //初始将链表置空
    scanf("%d", &x); //输入节点的数据域
    while(x != 9999){ //输入9999表示插入结束
        s = (LNode*)malloc(sizeof(LNode)); //申请新节点s作为插入的节点
        s->data = x; //将数据元素x放在待插入节点s的数据域
        r->next = s; //修改尾节点的next域
        r = s; //修改尾指针指向新的尾节点
        scanf("%d", &x);} //继续输入下一个节点的数据域
    r->next = NULL; //尾插结束后, 将尾节点的next域置空
    return L;
}
```



2. 双链表

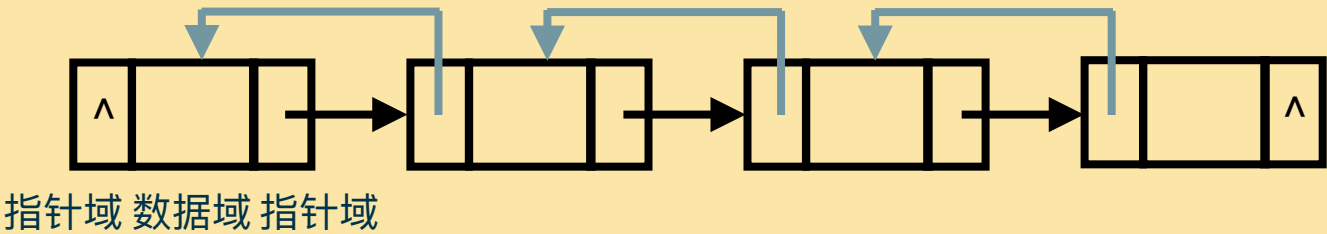


数据域 指针域

单链表的每个节点只有指向后继节点的指针
而在进行插入/删除操作时必须已知前驱节点的位置
此时只能通过遍历来定位，花费 $O(n)$ 时间



2. 双链表



数据域 存放数据元素的信息

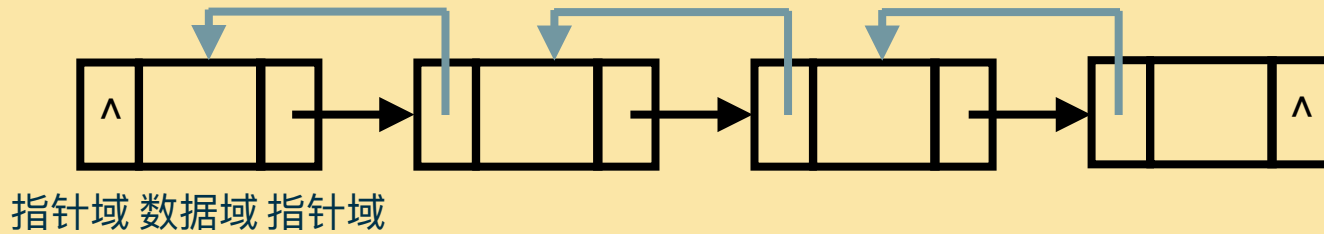
prior指针域 存放前驱节点的地址

next指针域 存放后继节点的地址

双链表通过为每个节点增加前驱指针，使得链表可以双向遍历
能在已知节点位置时以 $O(1)$ 时间完成删除与插入，弥补单链表必须遍历找前驱节点的不足



2. 双链表



```
typedef struct DNode{
    ElemType data; //节点的数据域
    struct DNode *prior, *next; //节点的指针域
}DNode, *DLinkedList;
```



2. 双链表

插入

```
s->prior = p->prior;  
p->prior->next = s;  
s->next = p;  
p->prior = s;
```



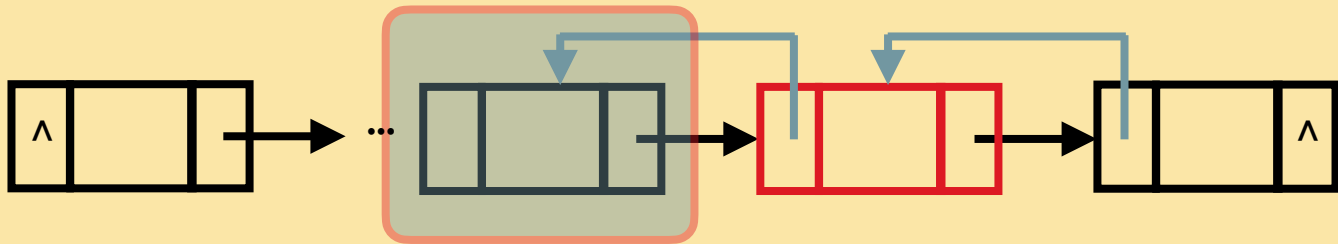
2. 双链表

删除

```
p->prior->next = p->next;  
p->next->prior = p->prior;  
free(p);
```



2. 双链表



删除某节点/在某位置插入节点

将工作指针移动到待删除/插入位置的前驱节点

双链表可以直接跟随prior指针定位前驱

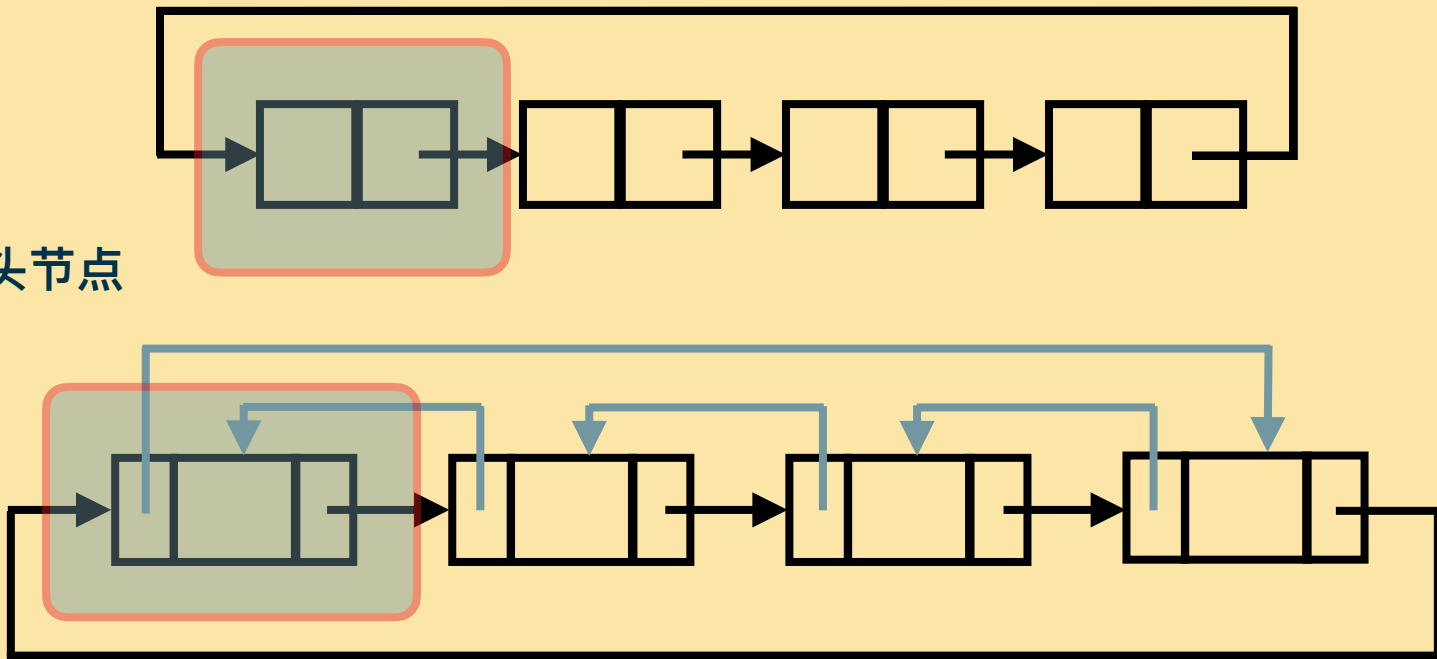
总结

双链表的优势就是找前驱方便
在已知插入/删除位置时
优先选择双链表



3. 循环链表

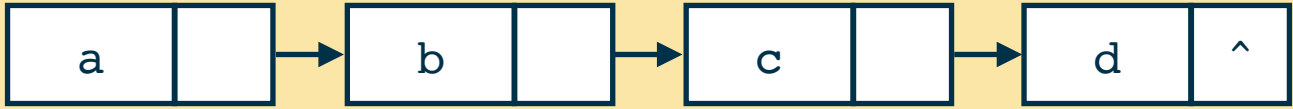
循环链表一般不设置头节点



循环链表把尾节点与头节点相连，使链表首尾等价
可从任意节点连续访问整个表，从而避免链表在表头表尾处的特殊处理



4. 静态链表



0		2
1	b	7
2	a	1
3	d	-1
4		
5		
6		
7	c	3

用数组下标代替next指针的单链表
需要预先分配一片连续空间

```
#define MaxSize 50 //静态链表的最大长度
typedef struct{
    ElemType data; //存储数据元素
    int next; //下一个元素的数组下标
}SLinkList[MaxSize];
```



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 已知头指针h指向一个带头节点的非空单循环链表，节点结构为

data	next
------	------

，其中next是指向直接后继节点的指针，p是尾指针，q是临时指针。现要删除该链表的第一个元素，正确的语句序列是

A. `h->next = h->next->next; q = h->next; free(q);`

B. `q = h->next; h->next = h->next->next; free(q);`

C. `q = h->next; h->next = q->next; if(p!=q) p=h; free(q);`

D. `q = h->next; h->next = q->next; if(p == q) p=h; free(q);`



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 已知头指针h指向一个带头节点的非空单循环链表，节点结构为

data	next
------	------

，其中next是指向直接后继节点的指针，p是尾指针，q是临时指针。现要删除该链表的第一个元素，正确的语句序列是

A. `h->next = h->next->next; q = h->next; free(q);`

B. `q = h->next; h->next = h->next->next; free(q);`

C. `q = h->next; h->next = q->next; if(p!=q) p=h; free(q);`

D. `q = h->next; h->next = q->next; if(p == q) p=h; free(q);`



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 已知表头元素为c的单链表在内存中的存储状态如下表所示

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将f存放于1014H处并插入到单链表中，若f在逻辑上位于a和e之间，则a，e，f的“链接地址”依次是

- A. 1010H, 1014H, 1004H
- B. 1010H, 1004H, 1014H
- C. 1014H, 1010H, 1004H
- D. 1014H, 1004H, 1010H



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 已知表头元素为c的单链表在内存中的存储状态如下表所示

地址	元素	链接地址
1000H	a	1010H
1004H	b	100CH
1008H	c	1000H
100CH	d	NULL
1010H	e	1004H
1014H		

现将f存放于1014H处并插入到单链表中，若f在逻辑上位于a和e之间，则a，e，f的“链接地址”依次是

- A. 1010H, 1014H, 1004H
- B. 1010H, 1004H, 1014H
- C. 1014H, 1010H, 1004H
- D. 1014H, 1004H, 1010H

祝你今天特别特别开心！